

Final Assignment — Event Booking Platform

1. Overview

This assignment is the final deliverable of your internship. You are expected to design and build a working backend system on your own, applying the knowledge and skills you have developed throughout the program. The scenario described below is intentionally realistic but not overly complex — the difficulty lies in building it correctly, not in the business logic itself.

2. Scenario

You will build the backend for EventBooking, a platform where organizers publish events taking place at venues, and attendees browse and reserve seats for the events that interest them.

The system supports three roles:

- ADMIN — manages venues and categories, manages user accounts, and has oversight over all bookings in the system.
- ORGANIZER — creates and manages the events they publish.
- ATTENDEE — browses events and books seats.

3. Core Domain

The system is built around the following core entities. You may introduce additional entities if you find it useful for your design (for example, a waitlist, a notification log, or a payment record), but the entities and fields listed below must remain part of your model as described.

User

`id, username, password, email, role`

Venue

`id, name, address, city, capacity`

Category

`id, name`

Event

`id, title, description, startDateTime, endDateTime, price, totalSeats, availableSeats, status`

Booking

`id, seatsBooked, status, bookingDate`

Review

`id, rating, comment, createdAt`

Business rules governing the relationships between entities

Rather than being told directly how to map these entities, you are expected to infer the correct relationship type from the business rules below:

- A venue can host many events, but each event takes place at exactly one venue.
- A user can organize many events, but each event is created and owned by exactly one organizer.
- An event can be associated with multiple categories, and a category can apply to multiple events.
- A user can make many bookings, but each individual booking is made by exactly one user.
- An event can have many bookings made against it, but each booking refers to exactly one event.
- A user can leave a review for an event only if they held a confirmed booking for that event and the event has already taken place. A user may leave at most one review per event.
- An event has a limited number of available seats, which must never be exceeded by the sum of active bookings against it.

You are responsible for translating these rules into a correctly designed data model.

4. Functional Requirements

As an unauthenticated visitor, you must be able to:

- Register a new account.
- Log in to receive access to the system.
- Browse and search published events — filtering by category, city, date range, and price range, with support for pagination and sorting.
- View the details of a single event, including its venue, organizer, categories, and average rating.

As an ATTENDEE, you must additionally be able to:

- Book one or more seats for a published event, as long as enough seats remain available.
- View all your own bookings, optionally filtered by status.
- Cancel one of your own bookings, subject to a cancellation policy that you must define and enforce (for example, bookings cannot be cancelled within a certain time window before the event starts). When a confirmed booking is cancelled and the event had no remaining seats, the next attendee on the event's waitlist (if you implement one) should be promoted to a confirmed booking.
- Join a waitlist for an event that is fully booked, as an alternative to booking directly.
- Leave a rating and review for an event you attended, once it has taken place.

As an ORGANIZER, you must additionally be able to:

- Create, update, publish, and cancel events that you own.
- View all bookings made against your own events.
- You must not be able to modify or cancel events belonging to another organizer.

As an ADMIN, you must additionally be able to:

- Manage venues and categories (create, update, remove).
- Manage user accounts, including activating or deactivating a user.
- View and cancel any booking in the system, regardless of who made it.
- View all events in the system, regardless of the organizer.

You are responsible for designing the API surface (resources, structure, request/response shapes) that exposes this functionality. No specific endpoint layout is prescribed — this is part of what is being evaluated.

5. Technical & Delivery Requirements

The following points must be respected regardless of how you choose to implement the functional requirements above:

- **Repository & access** — The project must be delivered in a private Git repository, with the mentors added as collaborators with access to it.
- **Configuration profiles** — The application must support multiple profiles (e.g. dev, int/testing, prod), each with its own configuration.
- **Project structure** — The project must be built as a single Maven module. If you have a good reason to prefer a multi-module structure, you may use one instead.
- **Data access** — Across your data access layer, you are expected to make use of every available way of querying the database at least once (derived query methods, JPQL, and native queries).
- **Authentication & authorization** — You must implement a login mechanism using JWT-based authentication. Access to functionality must be restricted according to the three roles described in Section 2 and Section 4 — each role should only be able to perform the actions listed for it.
- **Testing** — Automated tests are required and are expected to go beyond trivial unit tests of isolated components. Your test suite should also demonstrate integration testing across multiple components/services (for example, verifying that a booking request correctly flows through validation, persistence, and seat availability updates).
- **Documentation & logging** — API documentation (Swagger/OpenAPI) and application logging must be used appropriately throughout the project.
- **Code quality** — Clean code and best practices must be followed throughout the codebase. Design patterns are not required everywhere — apply them only where they genuinely fit the problem, not for their own sake.

6. Deliverables

- A private Git repository containing the full source code, with the mentors added as collaborators. The repo should contain organized commits with proper messages, not a single commit.
- A README.md explaining how to set up and run the project, including any environment variables or configuration required for each profile.
- Working API documentation accessible once the application is running.
- A test suite that can be executed via a standard Maven command.